

BudgetCollab

Application Collaborative de Gestion de Budget

Rapport de Soutenance

Présenté par : Hamdi Boutiti et Jlassi Moataz

Date : Mai 2026

Établissement : iTeam university

Encadrant : Saffar Hela

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Problématique	4
1.3	Objectifs	4
2	Analyse des besoins	6
2.1	Besoins fonctionnels	6
2.2	Besoins non fonctionnels	6
3	Conception et architecture	7
3.1	Architecture globale	7
3.2	Base de données	8
3.3	Couches logicielles	9
3.3.1	Core (noyau)	9
3.3.2	DAO	9
3.3.3	Services	9
3.3.4	Contrôleurs	9
3.4	Routage	9
4	Technologies utilisées	11
4.1	Langages	11
4.2	Bibliothèques	11
4.3	Environnement	11
5	Fonctionnalités implémentées	12
5.1	Authentification	12
5.2	Tableau de bord	12
5.3	Transactions	12
5.4	Budgets	13
5.5	Catégories	13
5.6	Alertes	13

5.7	Fonctionnalités collaboratives	13
5.8	Administration	13
6	Sécurité	14
7	Limitations et améliorations	15
8	Conclusion	16

Introduction

Contexte

Dans un environnement économique où la maîtrise des finances personnelles et collectives est devenue une nécessité, la gestion budgétaire occupe une place centrale dans la vie quotidienne des individus et des groupes. Les outils traditionnels — feuilles de calcul, relevés bancaires papier — présentent des limites importantes en termes de collaboration, de réactivité et de visibilité. Les solutions du marché sont souvent trop coûteuses, trop complexes, ou non adaptées aux besoins d’une famille, de colocataires ou d’une petite équipe.

C’est dans ce contexte qu’est né **BudgetCollab** : une application web collaborative de gestion de budget, conçue pour permettre à plusieurs utilisateurs de gérer ensemble leurs finances de manière simple, sécurisée et accessible sur un réseau local.

Problématique

La gestion budgétaire collective pose plusieurs défis spécifiques :

- Comment garantir que chaque membre du groupe visualise l’état des finances en temps réel ?
- Comment s’assurer que les alertes de dépassement soient transmises efficacement à tous ?
- Comment gérer les permissions et les rôles dans un environnement collaboratif sécurisé ?

Les applications bancaires classiques n’offrent pas de vue consolidée collaborative, et les tableurs partagés manquent d’automatisation et de sécurité.

Objectifs

1. Développer une application web sécurisée pour la gestion collaborative de budgets
2. Permettre le suivi en temps réel des transactions et des soldes
3. Offrir un système d’alertes automatiques aux seuils de 80 % et 100 %

4. Faciliter la collaboration via invitations et budgets partagés
5. Proposer une interface intuitive avec graphiques et visualisations
6. Assurer un niveau de sécurité conforme aux standards du web

Analyse des besoins

Besoins fonctionnels

Module d'authentification

Création de compte sécurisée, connexion, déconnexion, gestion du profil et changement de mot de passe. Validation des comptes par un administrateur.

Gestion des transactions CRUD complet des revenus et dépenses avec catégorisation et association à un budget.

Gestion des budgets Création avec plafond, période et type (individuel/partagé). Barre de progression visuelle.

Système d'alertes Déclenchement automatique à 80 % (vigilance) et 100 % (critique).

Tableau de bord KPIs, graphiques d'évolution et de répartition, filtres par période.

Fonctionnalités collaboratives Invitation par email, acceptation/refus, budgets partagés.

Panneau d'administration Validation des comptes, statistiques, gestion des utilisateurs.

Besoins non fonctionnels

Exigence	Description
Sécurité	Anti-injection SQL, XSS, CSRF ; bcrypt ; sessions sécurisées
Performance	Temps de réponse < 2 s pour les opérations courantes
Maintenabilité	Architecture MVC 3 couches (Contrôleur / Service / DAO)
Utilisabilité	Interface responsive, dark mode, icônes, navigation claire
Portabilité	Environnement LAMP/WAMP standard sans dépendances lourdes

TABLE 2.1 – Besoins non fonctionnels

Conception et architecture

Architecture globale

BudgetCollab adopte une **architecture 3 tiers** couplée au **MVC** :

- **Tier 1 — Frontend** : Vues PHP, CSS, JavaScript, Lucide Icons.
- **Tier 2 — Backend** : Contrôleurs (routage), Services (logique métier), DAO (accès données).
- **Tier 3 — Base de données** : MySQL 8 avec InnoDB, clés étrangères.

Le **Front Controller** (`public/index.php`) est le point d'entrée unique. Il initialise la session, charge la configuration, et dispatche via le routeur.

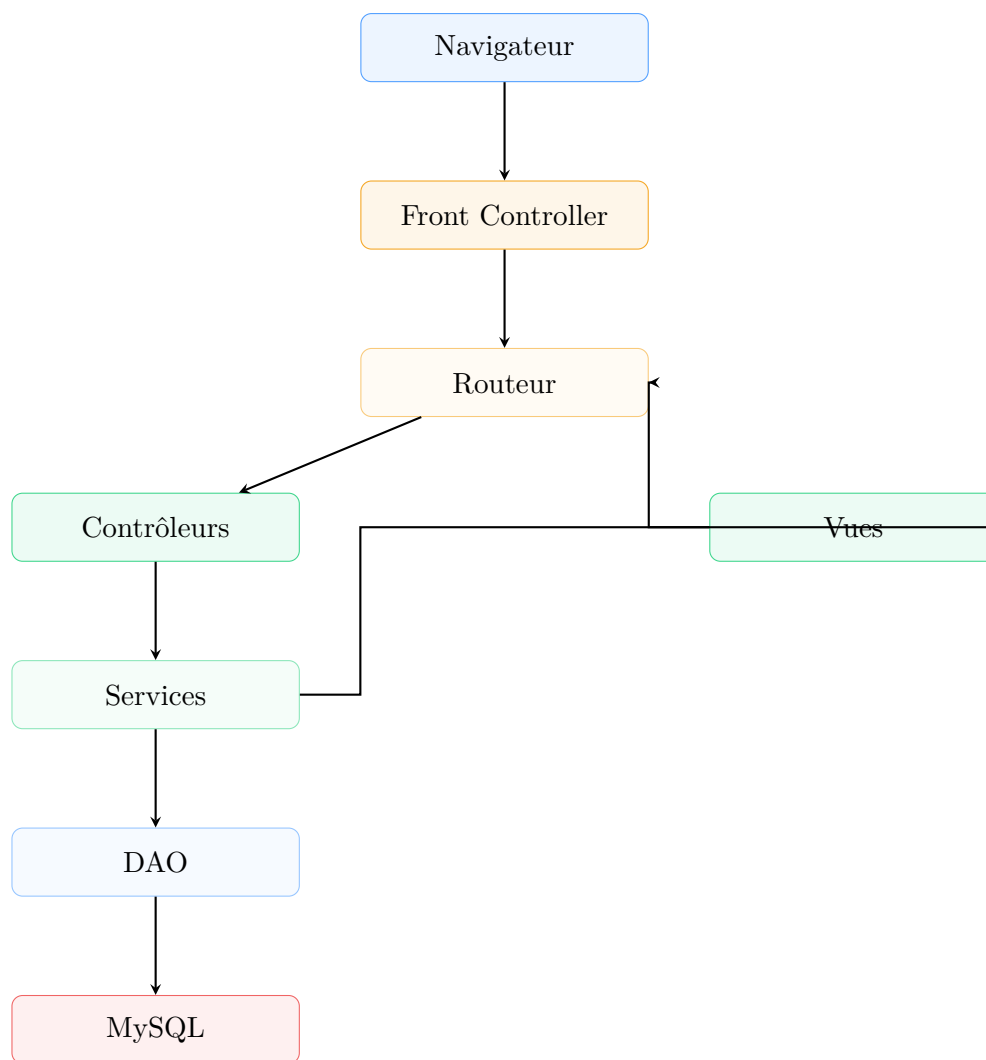


FIGURE 3.1 – Architecture de BudgetCollab

Base de données

Table	Rôle
users	Utilisateurs (nom, email, mdp_hash, rôle, statut)
categories	Catégories (nom, icône, est_default, user_id)
budgets	Budgets (nom, plafond, période, type)
budget_categories	Plafonds par catégorie pour chaque budget
budget_members	Membres (rôle, statut, token invitation)
transactions	Revenus et dépenses (montant, type, date)
alertes	Alertes (niveau, message, lue)

TABLE 3.1 – Tables de la base de données

Couches logicielles

Core (noyau)

Database (Singleton PDO), **Session** (sessions sécurisées), **Router** (mapping URL), **Response** (JSON/redirections), **Validator** (validation entrées).

DAO

UtilisateurDAO, TransactionDAO, BudgetDAO, CategorieDAO, AlerteDAO, MembreBudgetDAO.

Services

AuthService, TransactionService, BudgetService, DashboardService, AlerteService, AdminService.

Contrôleurs

AuthController, DashboardController, TransactionController, BudgetController, CategorieController, AlerteController, AdminController.

Routage

Méthode	Route	Action
GET	/	Dashboard
GET/POST	/login	Connexion
GET/POST	/register	Inscription
GET	/logout	Déconnexion
GET/POST	/profil	Profil utilisateur
POST	/profil/password	Changer mot de passe
GET	/dashboard	Tableau de bord
GET/POST	/transactions	Lister / Créer
POST	/transactions/{id}/update	Modifier
POST	/transactions/{id}/delete	Supprimer
GET/POST	/budgets	Lister / Créer
GET	/budgets/{id}	Détail budget
POST	/budgets/{id}/inviter	Inviter membre
GET	/mes-invitations	Voir invitations

Méthode	Route	Action
POST	/invitations/{id}/accepter	Accepter
POST	/invitations/{id}/refuser	Refuser
GET	/api/invitations/count	Compteur invitations
GET/POST	/categories	Catégories
GET	/api/categories	API JSON catégories
GET	/api/alertes	Alertes non lues
GET	/admin	Panneau admin

Technologies utilisées

Langages

PHP 8+ : orienté objet, PDO, bcrypt. **MySQL 8** : InnoDB, utf8mb4. **JavaScript ES6+** : Fetch API, DOM. **HTML5 + CSS3** : responsive, dark mode.

Bibliothèques

Chart.js 4.4 : graphiques (line, doughnut). **Lucide Icons** : icônes SVG vectorielles.

Environnement

Développement sous **XAMPP** (Apache + MySQL + PHP). Gestion de versions avec **Git/GitHub**.

Fonctionnalités implémentées

Authentification

Inscription avec validation (nom, email, mot de passe 8+ car.), hachage bcrypt (coût 12), statut EN_ATTENTE. Connexion vérifie statut ACTIF, régénère la session. CSRF token sur tous les formulaires.

Tableau de bord

KPIs (revenus, dépenses, solde, alertes). Graphiques Chart.js : courbes d'évolution, donut de répartition. Filtre par période (semaine/mois/année). Dernières transactions et budgets avec barres de progression.

Transactions

CRUD avec validation (type, montant positif, date, catégorie, budget). Recalcul automatique du solde du budget. Code couleur (vert revenu / rouge dépense).

Listing 5.1 – Ajout de transaction

```
1 public function ajouter(int $userId, array $dto): array {
2     $id = $this->txDao->save($userId, (int)$dto['budget_id'],
3         (int)$dto['categorie_id'], $dto['type'],
4         (float)$dto['montant'], $dto['date_op'],
5         $dto['description'] ?? '');
6     $taux = $this->budgetService->recalculer((int)$dto['
7         budget_id']);
8     return ['ok' => true, 'transaction_id' => $id, 'taux' =>
9         $taux];
10 }
```

Budgets

Création avec nom, plafond, période (MENSUEL/HEBDOMADAIRE/PERSONNALISE), type (INDIVIDUEL/PARTAGE). Barre de progression colorée (<80% vert, 80-99% orange, $\geq 100\%$ rouge). Invitation de membres par email.

Catégories

10 catégories par défaut (Alimentation, Transport, Logement,...). Création de catégories personnalisées. API JSON pour chargement dynamique.

Alertes

Déclenchement automatique : <80% : aucune alerte ; $\geq 80\%$: AVERTISSEMENT ; $\geq 100\%$: CRITIQUE. Badge de notification, marquage comme lu.

Fonctionnalités collaboratives

Invitations depuis la création du budget ou depuis la page de détail. Badge avec compteur dans la sidebar et la topbar (via `GET /api/invitations/count`). Page `/mes-invitations` avec acceptation/refus. Budgets partagés visibles par tous les membres.

Administration

Validation des comptes en attente. Gestion des rôles et statuts. Statistiques globales (utilisateurs, transactions, budgets).

Sécurité

Mesure	Implémentation
Injection SQL	100 % PDO prepared statements avec paramètres typés
XSS	<code>htmlspecialchars(\$, ENT_QUOTES)</code> sur toutes les sorties
CSRF	Token <code>bin2hex(random_bytes(32))</code> vérifié sur chaque POST
Mots de passe	<code>password_hash()</code> bcrypt coût 12, <code>password_verify()</code>
Sessions	<code>httponly</code> , <code>samesite=Lax</code> , régénération après connexion
Validation	Classe Validator : <code>required</code> , <code>email</code> , <code>numeric</code> , <code>date</code> , <code>inArray</code> , <code>minLength</code>

TABLE 6.1 – Mesures de sécurité

Limitations et améliorations

Prio	Amélioration	Description
Haute	Service email	PHPMailer pour notifications et invitations
Haute	Tests automatisés	PHPUnit pour services et DAO
Moyenne	Export PDF/CSV	Export des transactions
Moyenne	Pagination avancée	Filtres combinés et recherche
Moyenne	Objectifs financiers	Module d'épargne avec suivi
Basse	API REST	JWT pour application mobile
Basse	WebSockets	Notifications temps réel
Basse	2FA	Authentification à deux facteurs

TABLE 7.1 – Améliorations futures

Conclusion

Le projet **BudgetCollab** a permis de concevoir et développer une application web complète de gestion collaborative de budget.

Apports techniques : Architecture MVC 3 tiers, Front Controller, DAO Pattern, PHP orienté objet sans framework.

Sécurité : Prepared statements, échappement XSS, CSRF, bcrypt, sessions sécurisées, validation entrées.

Valeur ajoutée : Invitations, budgets partagés, alertes automatiques, badges de notification.

Perspectives : Service email, export, tests, API REST, application mobile.

Bilan

BudgetCollab est un projet abouti alliant rigueur architecturale, qualité de code, sécurité et expérience utilisateur, répondant à des besoins réels de gestion budgétaire collaborative.

Rapport généré le 29 mai 2026

https://github.com/Mooataz/budget_app